

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

PEKAN 3

MEMBUAT FUNGSI CRUD (CREATE, READ, UPDATE, DELETE) USER DENGAN DATABASE MYSQL (Lanjutan)



Oleh :

Jiyad Rifqi Pasaribu NIM 2411533003

MATA KULIAH PRAKTEKUM PEMROGRAMAN BERORIENTASI OBJEK

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

A. PENDAHULUAN

Di zaman digital seperti sekarang, mengatur data sangat penting di banyak bidang, seperti bisnis dan organisasi. Data pelanggan harus dikelola dengan baik supaya mudah untuk dicatat, diperbarui, atau dihapus jika sudah tidak diperlukan. Salah satu cara untuk mengatur data ini adalah dengan membuat aplikasi desktop sederhana.

Pada pembahasan ini dibuat menggunakan Java Swing untuk memudahkan pengguna dalam melakukan operasi CRUD (menambah, melihat, mengubah, dan menghapus) data pelanggan. Dengan tampilan yang mudah dipahami, pengguna bisa menambahkan data baru, melihat data dalam tabel, mengubah data yang sudah ada, dan menghapus data yang tidak dibutuhkan. Dengan program ini, pengelolaan data pelanggan menjadi lebih mudah, teratur, dan praktis.

B. TUJUAN

Tujuan dari praktikum ini adalah sebagai berikut:

1. Memberikan kemudahan dalam melakukan pengelolaan data pelanggan secara terstruktur melalui operasi CRUD (Create, Read, Update, Delete).
2. Menyediakan antarmuka aplikasi yang sederhana dan mudah digunakan sehingga pengguna dapat menambah, melihat, memperbarui, dan menghapus data pelanggan dengan cepat.
3. Meningkatkan efisiensi dalam pencatatan dan pemeliharaan data pelanggan tanpa harus dilakukan secara manual.

C. ISI

1. Membuat sistem create user pada bagian user frame dengan cara klik pada bagian tombol save.
2. Lalu klik kanan pada tombol tersebut pilih add event handler, actionPerformed, kemudian isi kode ini.

```
User user = new User();
user.setNama(nama);
user.setUsername(usernameFormatted);
user.setPassword(password);
usr.save(user);
reset();
loadTable();
}
```

Ketika tombol Save ditekan, program akan mengambil data dari form berupa nama, username, dan password. Username otomatis diformat menjadi huruf kecil tanpa spasi, kemudian password dicek agar panjangnya minimal enam karakter, jika tidak maka muncul pesan error. Jika validasi lolos, program membuat objek User baru, mengisinya dengan data form, lalu menyimpannya ke database melalui `usr.save(user)`. Setelah itu, form dikosongkan kembali menggunakan `reset()` dan tabel pengguna diperbarui dengan `loadTable()` agar data baru langsung tampil di layar.

3. Selanjutnya buatlah method dengan nama load table untuk menampilkan table yang berfungsi untuk mengambil data user dari database, memasukkannya ke model tabel, lalu menampilkan data tersebut di JTable.

```
public void loadTable() {
    ls = usr.show();
    TableUser tu = new TableUser(ls);
    tableUsers.setModel(tu);
    tableUsers.getTableHeader().setVisible(true);
}
```

4. Panggil method pada class main agar saat program di jalankan maka load table akan di panggil.

```
UserFrame frame = new UserFrame();
frame.setVisible(true);
```

5. Klik dua kali pada jTable yang ada pada userframe lalu tambahkan kodingan ini.

```

tableUsers = new JTable();
tableUsers.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        id = tableUsers.getValueAt(tableUsers.getSelectedRow(),0).toString();
        txtName.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),1).toString());
        txtUsername.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),2).toString());
        txtPassword.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),3).toString());
    }
});

```

Buat sebuah mouse listener di tabel tableUsers supaya setiap kali baris diklik, data dari baris itu langsung muncul di form input. Pertama, ambil baris yang dipilih dengan `getSelectedRow()`. Setelah itu, ambil nilai-nilai dari baris tersebut menggunakan `getValueAt(baris, kolom)`. Simpan data dari kolom pertama (indeks 0) ke variabel `id`. Kemudian, masukkan data dari kolom berikutnya ke dalam field teks yang sesuai: nama ke `txtName`, username ke `txtUsername`, dan password ke `txtPassword`. Dengan cara ini, saat pengguna memilih baris tertentu, data langsung tampil di form sehingga lebih mudah untuk mengubah atau menghapus data tersebut.

6. Coba tabel yang sudah di buat apakah inputan sudah masuk ke dalam tabel nya.

5	Jiyad	JiyadR	12345
6	rifqi	www.	hnb56
7	Juan Garcia	juangarcia	1345
8	burhan	burhansyah	1234568
9	zaki	zakir	123345

7. Klik dua kali pada tombol update dan tambahkan kodingan ini yang berfungsi untuk memperbarui data user yang dipilih dari tabel. Saat ditekan, program mengambil input dari form (nama, username, dan password), memformat username menjadi huruf kecil tanpa spasi, lalu menyimpan data tersebut ke objek `User` bersama dengan `id` dari user yang dipilih. Objek ini kemudian dikirim ke method `update()` pada `UserRepo` untuk mengubah data di database. Setelah itu, form dikosongkan kembali dan tabel diperbarui agar perubahan langsung terlihat.

```

JButton btnUpdate = new JButton("Update");
btnUpdate.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        User user = new User();
        user.setNama(txtName.getText());

        String usernameFormatted = txtUsername.getText().toLowerCase().replaceAll("\\s+", "");
        user.setUsername(usernameFormatted);

        user.setPassword(txtPassword.getText());
        user.setId(id);
        usr.update(user);
        reset();
        loadTable();
    }
});

```

8. Selanjutnya buat untuk method delete user dengan cara klik dua kali pada tombol delete dan isi untuk menghapus data user. Saat ditekan, program mengecek apakah variabel id berisi nilai (artinya ada baris tabel yang sudah dipilih). Jika ada, maka method delete(id) pada UserRepo dipanggil untuk menghapus user tersebut dari database. Setelah data terhapus, form dikosongkan dengan reset() dan tabel diperbarui dengan loadTable() agar perubahan langsung terlihat. Jika belum ada data yang dipilih (id == null), maka ditampilkan pesan peringatan agar pengguna memilih data terlebih dahulu sebelum menghapus.

```

JButton btnDelete = new JButton("Delete");
btnDelete.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if(id != null) {
            usr.delete(id);
            reset();
            loadTable();
        } else {
            JOptionPane.showMessageDialog(null, "Silahkan pilih data yang akan di hapus");
        }
    }
});

```

9. Selanjutnya buat tabel baru dengan nama pelanggan pada mysql kita

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 nama	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 telepon	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More

10. Lalu buat class baru dengan nama PelangganRepo di package DAO untuk menghubungkan kodingan kita ke database.

```
1 package DAO;
2
3 import java.sql.*;
4 import java.util.ArrayList;
5 import java.util.List;
6 import model.Pelanggan;
7
8 public class PelangganRepo {
9
10     private Connection getConnection() throws Exception {
11         String url = "jdbc:mysql://localhost:3306/laundry_apps";
12         String user = "root";
13         String password = "";
14         Class.forName("com.mysql.cj.jdbc.Driver");
15         return DriverManager.getConnection(url, user, password);
16     }
17
18     public void save(Pelanggan pelanggan) {
19         String sql = "INSERT INTO pelanggan (nama, email, telepon) VALUES (?, ?, ?)";
20         try (Connection conn = getConnection();
21             PreparedStatement stmt = conn.prepareStatement(sql)) {
22
23             stmt.setString(1, pelanggan.getNama());
24             stmt.setString(2, pelanggan.getEmail());
25             stmt.setString(3, pelanggan.getTelepon());
26             stmt.executeUpdate();
27
28             System.out.println("Data pelanggan berhasil disimpan ke database!");
29         } catch (Exception e) {
30             e.printStackTrace();
31         }
32     }
33
34     public List<Pelanggan> show() {
35         List<Pelanggan> list = new ArrayList<>();
36         String sql = "SELECT * FROM pelanggan";
37         try (Connection conn = getConnection();
38             Statement stmt = conn.createStatement();
39             ResultSet rs = stmt.executeQuery(sql)) {
40
41             while (rs.next()) {
42                 Pelanggan p = new Pelanggan();
43                 p.setId(rs.getString("id")); // pastikan kolom id ada di tabel
44                 p.setNama(rs.getString("nama"));
45                 p.setEmail(rs.getString("email"));
46
47                 list.add(p);
48             }
49         } catch (Exception e) {
50             e.printStackTrace();
51         }
52         return list;
53     }
54
55     public void update(Pelanggan pelanggan) {
56         String sql = "UPDATE pelanggan SET nama=?, email=?, telepon=? WHERE id=?";
57         try (Connection conn = getConnection();
58             PreparedStatement stmt = conn.prepareStatement(sql)) {
59
60             stmt.setString(1, pelanggan.getNama());
61             stmt.setString(2, pelanggan.getEmail());
62             stmt.setString(3, pelanggan.getTelepon());
63             stmt.setString(4, pelanggan.getId());
64             stmt.executeUpdate();
65
66             System.out.println("Data pelanggan berhasil diupdate di database!");
67         } catch (Exception e) {
68             e.printStackTrace();
69         }
70     }
71
72     public void delete(String id) {
73         String sql = "DELETE FROM pelanggan WHERE id=?";
74         try (Connection conn = getConnection();
75             PreparedStatement stmt = conn.prepareStatement(sql)) {
76
77             stmt.setString(1, id);
78             stmt.executeUpdate();
79         }
80     }
81 }
```

```

1 public void delete(String id) {
2     String sql = "DELETE FROM pelanggan WHERE id=?";
3     try (Connection conn = getConnection();
4         PreparedStatement stmt = conn.prepareStatement(sql)) {
5
6         stmt.setString(1, id);
7         stmt.executeUpdate();
8
9         System.out.println("Data pelanggan berhasil dihapus dari database!");
10    } catch (Exception e) {
11        e.printStackTrace();
12    }
13 }
14 }
15 }

```

Kode di atas adalah program Java berbasis Swing yang membuat tampilan form manajemen data pelanggan dengan fitur CRUD (Create, Read, Update, Delete). Program menampilkan input teks untuk nama, email, dan telepon, serta tabel untuk menampilkan daftar pelanggan. Tombol Save digunakan untuk menambah data baru ke repository, Update untuk memperbarui data yang dipilih dari tabel, Delete untuk menghapus data dengan konfirmasi, dan Cancel untuk mengosongkan form. Data pelanggan ditampilkan di tabel menggunakan TablePelanggan, dan ketika pengguna mengklik salah satu baris tabel, data tersebut otomatis ditampilkan ke input sehingga bisa diubah atau dihapus.

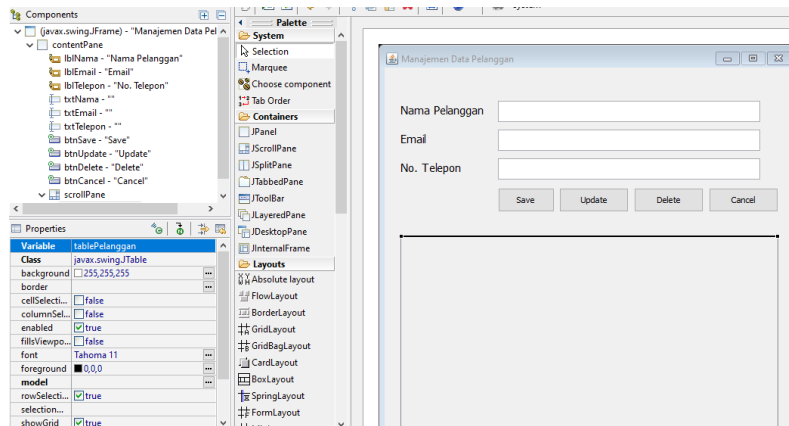
11. Buat juga class baru di package tabel dengan nama TablePelanggan untuk

```

1 package tabel;
2
3 import java.util.List;
4 import javax.swing.table.AbstractTableModel;
5 import model.Pelanggan;
6
7 public class TablePelanggan extends AbstractTableModel {
8
9     private List<Pelanggan> list;
10
11     public TablePelanggan(List<Pelanggan> list) {
12         this.list = list;
13     }
14
15     @Override
16     public int getRowCount() {
17         return list.size();
18     }
19
20     @Override
21     public int getColumnCount() {
22         return 4;
23     }
24
25     @Override
26     public Object getValueAt(int rowIndex, int columnIndex) {
27         switch (columnIndex) {
28             case 0: return list.get(rowIndex).getId();
29             case 1: return list.get(rowIndex).getNama();
30             case 2: return list.get(rowIndex).getEmail();
31             case 3: return list.get(rowIndex).getTelepon();
32             default: return null;
33         }
34     }
35
36     @Override
37     public String getColumnName(int column) {
38         switch (column) {
39             case 0: return "ID";
40             case 1: return "Nama Pelanggan";
41             case 2: return "Email";
42             case 3: return "No. Telepon";
43             default: return null;
44         }
45     }
46 }

```

12. Terakhir untuk bisa menampilkan pelanggan buat JFrame dengan nama PelangganFrame.



13. Buat lagi tabel baru dengan nama service pada mysql.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 service_name	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 description	text	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 price	double			No	None			Change Drop More

14. Sama dengan sebelumnya, buat class baru dengan nama ServiceRepo di package DAO untuk menghubungkan dengan database.

```

1 package DAO;
2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7 import java.util.ArrayList;
8 import java.util.List;
9 import model.Service;
10 import config.Database;
11
12 public class ServiceRepo {
13
14     public void save(Service service) {
15         String sql = "INSERT INTO services (service_name, description, price) VALUES (?, ?, ?)";
16         try (Connection conn = Database.koneksi();
17              PreparedStatement pstmt = conn.prepareStatement(sql)) {
18             pstmt.setString(1, service.getServiceName());
19             pstmt.setString(2, service.getDescription());
20             pstmt.setDouble(3, service.getPrice());
21             pstmt.executeUpdate();
22         } catch (SQLException e) {
23             e.printStackTrace();
24         }
25     }
26
27     public List<Service> show() {
28         List<Service> services = new ArrayList<>();
29         String sql = "SELECT * FROM services";
30         try (Connection conn = Database.koneksi();
31              PreparedStatement pstmt = conn.prepareStatement(sql);
32              ResultSet rs = pstmt.executeQuery()) {
33             while (rs.next()) {
34                 Service service = new Service();
35                 service.setId(rs.getString("id"));
36                 service.setServiceName(rs.getString("service_name"));
37                 service.setDescription(rs.getString("description"));
38                 service.setPrice(rs.getDouble("price"));
39                 services.add(service);
40             }
41         } catch (SQLException e) {
42             e.printStackTrace();
43         }
44         return services;
45     }
46 }

```



```

    }

    public void update(Service service) {
        String sql = "UPDATE services SET service_name = ?, description = ?, price = ? WHERE id = ?";
        try (Connection conn = Database.koneksi();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, service.getServiceName());
            pstmt.setString(2, service.getDescription());
            pstmt.setDouble(3, service.getPrice());
            pstmt.setString(4, service.getId());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void delete(String id) {
        String sql = "DELETE FROM services WHERE id = ?";
        try (Connection conn = Database.koneksi();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

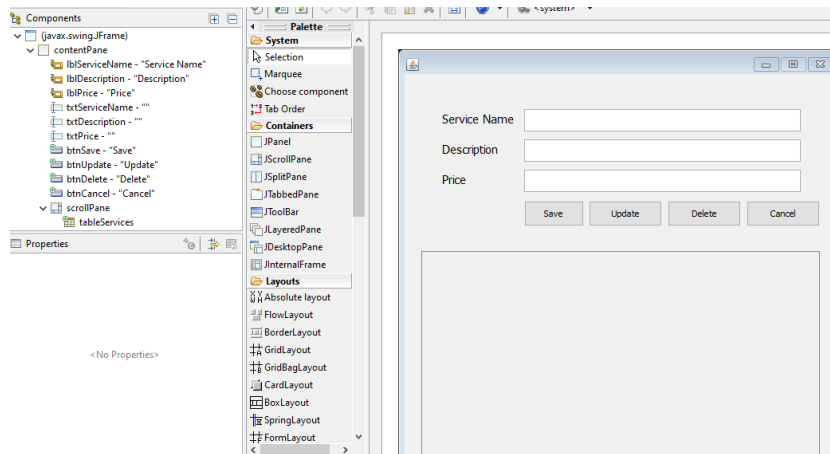
15. Buat juga class baru di package tabel dengan nama TableService untuk

```

1 package tabel;
2
3 import java.util.List;
4 import javax.swing.table.AbstractTableModel;
5 import model.Service;
6
7 public class TableService extends AbstractTableModel {
8     private List<Service> services;
9     private final String[] columnNames = {"ID", "Service Name", "Description", "Price"};
10
11     public TableService(List<Service> services) {
12         this.services = services;
13     }
14
15     @Override
16     public int getRowCount() {
17         return services.size();
18     }
19
20     @Override
21     public int getColumnCount() {
22         return columnNames.length;
23     }
24
25     @Override
26     public String getColumnName(int column) {
27         return columnNames[column];
28     }
29
30     @Override
31     public Object getValueAt(int rowIndex, int columnIndex) {
32         Service service = services.get(rowIndex);
33         switch (columnIndex) {
34             case 0:
35                 return service.getId();
36             case 1:
37                 return service.getServiceName();
38             case 2:
39                 return service.getDescription();
40             case 3:
41                 return service.getPrice();
42             default:
43                 return null;
44         }
45     }
46 }

```

16. Buat juga JFrame di package ui untuk menampilkan service.



D. KESIMPULAN

Dari hasil perancangan dan pembuatan program manajemen data pelanggan menggunakan Java Swing, dapat disimpulkan bahwa aplikasi ini berhasil memudahkan pengelolaan data pelanggan dengan cara yang simpel namun efisien. Fitur CRUD (Create, Read, Update, Delete) memungkinkan pengguna untuk menambah, melihat, mengedit, dan menghapus data pelanggan lewat antarmuka yang user-friendly. Program ini juga memperlihatkan penerapan konsep pemrograman berorientasi objek serta penggunaan komponen GUI di Java untuk membangun aplikasi desktop yang berguna. Secara keseluruhan, aplikasi ini menjadi solusi yang praktis untuk pengelolaan data sekaligus sarana pembelajaran dalam mengembangkan sistem informasi yang lebih kompleks ke depannya.